

5-Minute Demo Script

Transformer WAF + Robust Phishing URL Detection Under Incomplete Information

September 2026

Before recording

1. Start the service: `./scripts/setup_and_run.sh` (Windows: `scripts\setup_and_run.bat`).
Wait for `Application startup complete` .
2. Open **`http://localhost:8000`** in a browser at full width. Confirm the top-right API key box shows `.....` (it is pre-filled with `dev-key`).
3. Optional: open `docs/EXAMPLES.md` on a second screen for the URLs.
4. Every number you will show is computed live or read from the database — nothing on screen is typed in by hand, so you can safely take questions.

Timing below adds up to about 5 minutes. Text in *italics* is what to say; **bold** is what to click.

0:00 – 0:40 · Overview page (/)

Show the landing page without scrolling.

“Transformer WAF — stop threats before they reach you. Attack traffic on the left, the security shield in front of the network globe, flagged threats around it: malicious URLs, phishing attempts, suspicious domains.”

Point at the terminal panel in the visual.

“This terminal is live: a path-traversal request scored by the WAF model when the page loaded — anomaly score above the threshold, flagged as a threat.”

Point at the metrics strip.

“Live from the database: URLs analysed, threats detected, the WAF anomaly rate and the mean inference time.”

0:40 – 1:10 · How it works and live analysis (scroll down on /)

Scroll to “Complete → remove → mitigate → compare”.

“The four steps of the experiment: baseline on complete information, remove segments on the same test URLs, train a model with incomplete examples, compare.”

Scroll to “Complete → missing → mitigated”.

“The same legitimate URL three times, live: complete — legitimate; with the subdomain and scheme removed and no handling, the baseline model says phishing; with the mitigation, legitimate again.”

Scroll to “Robustness under missing information”.

“Full information 99.8 %; remove half the segments and the baseline drops to 69 %; the mitigated model recovers to 90 %. The chart and table are read from the stored experiment — I’ll open it on the Research page.”

1:10 – 2:30 · Analyze page (/analyze)

Click Analyze in the top bar.

Step A — complete information. In **Load an example** choose `http://secure-login.paypal-verify.tk/account/update.php?id=99` . It fills the URL and pre-ticks *domain* and *scheme*; **untick both** for now. **Click Analyze URL.**

“Phishing, probability above 99 %. Below the verdict: the exact text the model saw, the seven segments, and the WAF anomaly score of the equivalent HTTP request — the original firewall model, reported separately because it answers a different question.”

Step B — missing information, no handling. In **Load an example** choose `https://www.readersdigest.co.uk` (a legitimate URL from the test split). It pre-ticks *subdomain* and *scheme*. Keep strategy **None (blank the segment)**. **Click Analyze URL.**

“Legitimate URL, but with www and https unavailable the baseline model says phishing at 99.97 %. Notice the text the model saw: just readersdigest.co.uk.”

Step C — same input, mitigation. Select **Training with incomplete examples**. **Click Analyze URL.**

“Same masked input, robust model: legitimate, 7 %. The [MISSING] markers are highlighted in the model text — the model was trained to cope with them.”

Step C' (one click instead of B + C). With the same URL and ticks, **click Run experiment**: the research story appears as a vertical flow — ground truth → complete information → segments removed / baseline → same input / robust model — all real inferences.

Step C'' — five levels. Click Missing-level tests: the same URL at 0 %, 10 %, 30 %, 50 % missing with the baseline, and 50 % with the mitigation (seeded, reproducible). For wikipedia.org: legitimate, legitimate, phishing, phishing, legitimate — the whole research result in one table.

Step D (optional, 15 s) — the honest failure case. Choose `https://www.github.com/login`, untick everything, **Analyze URL**.

“This is classified phishing although it is legitimate. In the training data no legitimate URL has a path, so the model learned ‘path means phishing’. That is a dataset limitation, documented in the README, and it is why the pipeline accepts any other labeled dataset.”

2:30 – 4:00 · Research page (/research)

Click Research in the top bar. **Click the Research overview tab** first.

“Problem, question, dataset, baseline model, methodology, mitigation, metrics and a conclusion generated from the latest stored experiment.”

Click the Missing-information sweep tab.

“Here is the experiment itself. Levels 0 to 50 % — each segment of each test URL is independently unavailable with that probability; a seed makes it reproducible. Three strategies are compared on identical masked inputs.”

Leave the defaults (all levels, all three strategies, 5,000 URLs, seed 42). **Click Run experiment**. It takes 5–10 seconds.

Point at the tiles, then the chart.

“Complete information 99.8 %. Grey line — no handling — falls to about 69 %. Blue — a marker the model never saw — is worse. Green — the mitigation — stays above 89 %. The interpretation text under the chart is generated from these numbers only.”

Change the metric dropdown to **F1**, then back to **accuracy**.

Scroll to “All conditions”.

“Precision, recall, F1, false-positive and false-negative rate, ROC AUC, and the change versus the complete reference for every condition. Without handling the damage is false positives — FPR rises to over 40 %. The provenance block records the experiment id, seed, dataset hash and which model checkpoints were used.”

Click the Feature dependency tab. Click Run ablation.

“Each segment removed on its own. Subdomain costs the baseline 57 points, scheme 15; domain and TLD almost nothing. That tells us what the model actually relies on — and the mitigated column shows the robust model barely needs the subdomain any more.”

Click the Results tab.

“Baseline metric cards, the full comparison table with precision, recall, F1, FPR, FNR and AUC for every condition, and the feature-dependency table.”

Click the Stored experiments tab.

“Every run is persisted. This one was on the full 47,072-URL test split — the numbers in the report. Open reloads it with chart and tables.”

4:00 – 4:40 · History page (</history>)

Click History.

“The original WAF functionality is intact: every scored request is stored, with filters and replay through the current model.”

Click Replay on any row → the blue notice shows the re-scored result.

Click the URL analyses tab.

“And the URL analyses we just did, including which segments were unavailable and which strategy was used.”

4:40 – 5:00 · Close

Click API (Swagger) briefly, or go back to the Overview page.

“Everything runs in one FastAPI service: the existing WAF endpoints, the new /detect/url and /experiment endpoints, the SQLite persistence and this UI. The research question is answered with measurement, not assumption: missing information degrades detection sharply, an explicit marker alone does not help, and training with incomplete examples recovers most of the loss.”

If something goes wrong during the demo

Problem	Do this
“Cannot reach the API”	the service stopped — rerun <code>scripts/setup_and_run.sh</code> , reload the page
401 in the UI	type <code>dev-key</code> into the API key box (top right)
Run experiment is slow	reduce <i>Test URLs</i> to 2000; results are still real, just on a smaller sample
A page looks unstyled	hard-refresh (Cmd/Ctrl+Shift+R) to clear a cached stylesheet
Want the light theme	click the sun icon in the top bar (the choice is remembered)